1 绪论

本章围绕两件事展开：区分数据结构中的基本概念；根据程序的执行过程分析时间与辅助空间。先理解定义和条件，再用少量代码验证，避免只背复杂度结论。

1.1 数据结构：对象、关系与操作

定义 1.1.1 (数据元素与数据项) 数据是信息的载体。**数据元素**是数据的基本单位，一个元素可由若干**数据项**组成；数据项是构成数据元素的不可分割的最小单位。**数据对象**是性质相同的数据元素的集合。

例如，学生记录是数据元素，学号、姓名、成绩是其中的数据项；一批同类学生记录组成数据对象。讨论数据结构时，还要说明这些元素之间的关系。

三要素　数据结构的三要素：逻辑结构、存储结构、数据的运算。逻辑结构描述元素之间的关系；存储结构把这些关系表示在计算机中；运算规定可以对数据做什么。

值与关系

存储时不仅保存元素值，还要体现元素之间的关系。关系可以由连续位置隐含，也可以由指针显式记录。

逻辑结构	元素关系	典型对象
集合	只同属一个集合	无先后关系的对象集合
线性结构	一对一的先后关系	线性表、栈、队列
树形结构	一对多的层次关系	树、二叉树
图状结构	多对多的一般关系	图

在线性结构中，除首元素外，每个元素有唯一直接前驱；除尾元素外，每个元素有唯一直接后继。一个元素也可以同时是首元素和尾元素。

存储方式	如何体现关系
顺序存储	逻辑相邻的元素保存在相邻存储位置
链式存储	用结点内的指针连接逻辑相关的结点
索引存储	建立索引项，记录定位信息
散列存储	根据关键字计算存储位置

关系与实现　逻辑结构与存储结构不能混为一谈。线性表可以顺序存储，也可以链式存储；采用链式存储，也不能据此判定对象一定是一棵树。

抽象数据类型（ADT）描述数据对象、数据关系和基本操作，不限定具体存储实现。以线性表为例，“取第 i 个元素”是运算定义；用数组直接访问或沿链表查找，是两种实现。

1.2 算法的特性与设计要求

定义 1.2.1 (算法) 算法是求解一类问题的有限操作序列。对合法输入，应在有限步骤内结束，每一步含义明确且能够执行。

特性	准确含义
有穷性	对每个合法输入，执行有限步后结束
确定性	每条指令含义明确，不产生歧义
可行性	每一步可用已实现的基本操作有限次完成
输入	有零个或多个输入
输出	有一个或多个输出

正确性、可读性、健壮性以及时间与空间效率是算法设计的重要评价要求。可行性不等于运行得足够快；有穷性也不等于执行次数很少。评价算法不能只看程序在某台计算机上运行了多少秒。

例 1.2.1 (边界条件（补充例）) 扫描数组求最大值时，若用 `max=A[0]` 初始化，就必须满足 $n \geq 1$。允许空数组时，应先处理空输入，不能访问不存在的首元素。

输出是什么

输出可以是返回值、修改后的数据状态等，不限于屏幕打印。

1.3 时间复杂度：先计数，再判断数量级

以输入规模 n 为变量，选择比较、赋值或数组访问等基本操作，统计其执行次数，再考察增长数量级。本章采用定长基本操作耗时为常数的模型。

记号与化简 $O(f(n))$ 表示渐近上界，$\Theta(f(n))$ 表示紧确数量级。408 选择题通常用 O 表达所要求的紧数量级；例如 $3n^2+7n+9=\Theta(n^2)$，答题时通常写 $O(n^2)$。

常见增长次序为：$1 \prec \log n \prec n \prec n\log n \prec n^2 \prec n^3 \prec 2^n \prec n!$。这里表示渐近增长快慢，不是某个固定 n 下的运行时间排序。

程序结构	分析方法
顺序执行的两个片段	次数相加，最终保留增长最快的项
条件分支	按具体输入分析；求最坏情况时取最坏分支
固定范围的嵌套循环	在每轮内层次数相同的条件下相乘
内层范围依赖外层	逐轮写出内层次数，再求和

变化或次数	由 k 得到的关系	数量级
每轮加 1，直到 n	$k \approx n$	$O(n)$
每轮乘 2，直到 n	$2^k \approx n$	$O(\log n)$
累计 $1+2+\cdots+k$	$k(k+1)/2 \approx n$	$O(\sqrt{n})$
内层依次执行 $1,2,\ldots,n$	$\sum_{i=1}^{n} i = n(n+1)/2$	$O(n^2)$

计数的通用方法

设循环执行 k 轮，把关键变量表示成 k 的函数，再代入终止条件。

1.3.1 累计和控制终止条件

【2017 年统考】下列函数的时间复杂度是（ ）。

题目代码

```
int func(int n) {
    int i = 0, sum = 0;
    while (sum < n) sum += ++i;
    return i;
}
```

A. $O(\log n)$ B. $O(n^{1/2})$ C. $O(n)$ D. $O(n\log n)$

解析 执行 k 轮后，$i=k$，$sum=k(k+1)/2$。循环结束时，k 是使 $k(k+1)/2\ge n$ 成立的最小非负整数，故 $k=\Theta(\sqrt{n})$，选 B。**判断次数要看每轮如何增长，不能把最终的 $sum\approx n$ 当成执行了 n 次。**

1.3.2 两层循环也可能是线性时间

【2022 年统考】下列程序段的时间复杂度是（ ）。

题目程序段

```
int sum = 0;
for (int i = 1; i < n; i *= 2)
    for (int j = 0; j < i; ++j)
        ++sum;
```

A. $O(\log n)$ B. $O(n)$ C. $O(n\log n)$ D. $O(n^2)$

解析 当 $n>1$ 时，各轮内层次数为 $1,2,4,\dots,2^k$，其中 $2^k<n\le 2^{k+1}$。总次数 $1+2+\cdots+2^k=2^{k+1}-1=\Theta(n)$，选 B。把每轮都按内层最大次数 n 估计，会得到不紧的上界。

1.3.3 最好、最坏与平均情况

同样长度为 n 的无序表，顺序查找在首位命中时只比较 1 次，最坏需要比较 n 次。最好和最坏分别取同规模合法输入中的最小、最大代价；平均情况还必须说明概率分布。

若保证查找成功，且每个元素被查找的概率相同，平均比较次数为 $(1+2+\cdots+n)/n=(n+1)/2$，仍是 $O(n)$。平均代价一般不能直接取最好与最坏的算术平均。

1.4 辅助空间：看同时保留多少状态

定义 1.4.1 (辅助空间) 辅助空间复杂度衡量算法额外使用的空间随输入规模增长的情况，通常不计输入本身。输出区域是否计入，按题目约定处理。

场景	辅助空间	原因
少量循环变量	$O(1)$	数量不随 n 增长
额外申请 n 个单元	$O(n)$	需同时保留 n 个单元
递归深度 n，每层常数空间	$O(n)$	调用栈尚未返回
非递归函数依次调用 n 次	$O(1)$*	前次返回后可复用

* 最后一行假设每次调用只用常数局部空间，且没有把结果累积保存到增长的容器中。

例如递归求阶乘，每次调用只做常数工作，但调用尚未返回时，参数、返回位置等状态要保留。按题目常用的单位成本模型，时间为 $O(n)$，递归栈空间也为 $O(n)$。**总调用次数影响时间，最大同时调用深度影响栈空间**。

原地算法通常只允许常数辅助空间。数组在原位置修改，并不自动意味着空间是 $O(1)$；若使用递归，仍要计入调用栈。

常数空间

$O(1)$ 不等于不用空间，而是额外空间不随输入规模增长。

检查是否理解条件

基础自检（补充题）

1. 线性表是否必须连续存储?

2. 每轮把变量乘 2，直到达到 n，需要多少轮?

3. 循环执行 n 次，是否就一定使用 $O(n)$ 辅助空间?

答案：1. 否，线性表描述逻辑关系，也可以链式存储。2. $O(\log n)$ 轮。3. 否，空间看同时占用的额外存储，几个循环变量仍是 $O(1)$。

2 线性表

本章重点是线性表的基本概念、两种存储实现和基本操作。所有代码采用 C++17 写法：位序从 1 开始，数组下标从 0 开始；普通单链表带有效头结点，尾结点的 next 为空。

2.1 定义、位序与基本操作

定义 2.1.1 (线性表) 线性表是具有相同数据类型的 n 个数据元素的有限序列。$n=0$ 时为空表；非空表的首元素没有直接前驱，尾元素没有直接后继，其余元素各有唯一直接前驱和直接后继。

“相同数据类型”不要求数值互不相同；“有限序列”同时强调元素数量有限和次序确定。只有一个元素时，该元素既没有直接前驱，也没有直接后继。

基本操作	含义或条件
初始化、判空、求长	建立空表；判断 $n=0$；取得元素个数
按位序取值	取第 i 个元素，$1 \le i \le n$
按值查找	查找满足给定值条件的元素
插入	在第 i 个位置插入，$1 \le i \le n+1$
删除	删除第 i 个元素，$1 \le i \le n$
销毁	释放动态申请的存储空间

位序与下标　第 i 个元素对应数组下标 $i-1$。长度为 n 的表有 $n+1$ 个合法插入位置，却只有 n 个合法删除位置。插入第 $n+1$ 位表示追加到表尾。

失败标记

按位序返回时，可用 0 表示查找失败；若按数组下标返回，应使用-1 等独立标记，避免与下标 0 混淆。

2.2 顺序表：连续存储与元素移动

顺序表用一段连续存储空间依次保存元素。若首元素地址为 b，每个元素占 w 个地址单位，第 i 个元素地址为 $b+(i-1)w$，因此可以在 $O(1)$ 时间按位序访问。

按位序与按值　随机存取指按给定位序直接定位。无序顺序表按值查找，仍然要比较元素，最坏时间为 $O(n)$。

顺序表的静态表示

```cpp
constexpr int MaxSize = 100;
struct SqList {
    int data[MaxSize];
    int length = 0;
};
```

`length` 是有效元素个数，`MaxSize` 是容量，始终满足 $0 \le length \le MaxSize$。静态数组容量固定，表长可以变化；动态分配的数组仍需保

持有效元素连续，扩容时可能搬移原有元素。

按值查找：返回第一个匹配元素的位序，失败返回 0

```
int LocateElem(const SqList &L, int x) {
    for (int j = 0; j < L.length; ++j)
        if (L.data[j] == x) return j + 1;
    return 0;
}
```

2.2.1 插入：先从后向前移动

在第 i 位插入 x，要先将原第 i 到第 n 个元素各后移一位，空出 `data[i-1]`。移动必须从表尾开始，避免覆盖尚未搬走的元素。

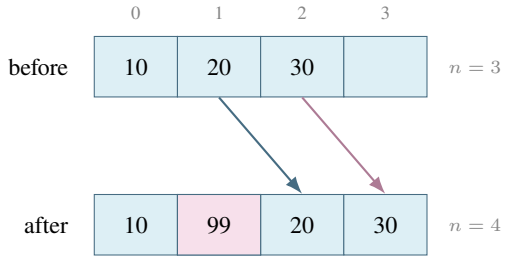


图 2.1: 在第 2 位插入 99：先把 30 移到下标 3，再把 20 移到下标 2，最后写入 99。

按位序插入：空间不足或位序非法时不修改表

```
bool ListInsert(SqList &L, int i, int x) {
    if (i < 1 || i > L.length + 1
        || L.length == MaxSize) return false;
    for (int j = L.length; j >= i; --j)
        L.data[j] = L.data[j - 1];
    L.data[i - 1] = x;
    ++L.length;
    return true;
}
```

移动元素数为 $n - i + 1$。最好情况为尾插，时间 $O(1)$；最坏为头插，时间 $O(n)$。若 $n+1$ 个位置等概率选择，平均移动次数为 $\dfrac{n + (n-1) + \cdots + 0}{n+1} = \dfrac{n}{2}$，平均时间为 $O(n)$。

插入边界

空表只能在第 1 位插入。尾插不移动元素；头插移动全部 n 个元素。这里分析的数组已有可用容量。

2.2.2 删除：保存元素，再从前向后移动

删除第 i 个元素时，先保存 `data[i-1]`，再把原第 $i+1$ 至第 n 个元素依次前移，最后表长减 1。有效区之外的旧数值不影响表的逻辑内容。

按位序删除：x 返回被删除的值

```
bool ListDelete(SqList &L, int i, int &x) {
    if (i < 1 || i > L.length) return false;
    x = L.data[i - 1];
    for (int j = i; j < L.length; ++j)
        L.data[j - 1] = L.data[j];
    --L.length;
    return true;
}
```

移动元素数为 $n-i$。尾删为 $O(1)$，头删最坏为 $O(n)$。若 n 个已有元素等概率被删除，平均移动次数为 $\dfrac{(n-1)+(n-2)+\cdots+0}{n}=\dfrac{n-1}{2}$。

例 2.2.1 (移动次数（补充例）) 长度为 8 的顺序表，在第 3 位插入需移动 $8-3+1=6$ 个元素；删除原表第 3 个元素需移动 $8-3=5$ 个元素。两种操作的合法位置数不同，平均移动次数的分母也不同。

2.3 单链表：结点、查找与链接修改

单链表的结点包含数据域和后继指针域。不同结点之间不要求地址连续，逻辑次序由 `next` 决定；同一个结点作为整体分配，其内部存储单元是连续的。

结点类型与带头结点的空表

```
struct Node {
    int data;
    Node *next;
};
// Initialize a list with a dummy head.
Node *L = new Node{0, nullptr};
```

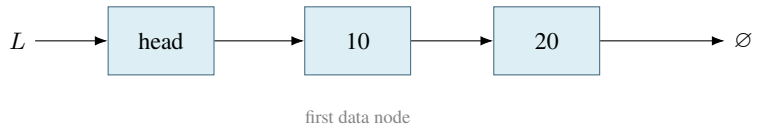


图 2.2: L 是头指针，指向头结点；首元结点为 L->next。头结点不保存有效表元素。

名称	指的是什么
头指针	保存链表起始结点地址的指针变量
头结点	为统一操作而附加的结点，不计入表长
首元结点	保存第一个有效数据元素的结点

带头结点后，首位置的插入和删除也有统一的前驱结点。普通带头结点单链表为空时，`L` 仍指向有效头结点，满足 `L->next == nullptr`。无头结点的空表则通常用头指针为空表示。

2.3.1 按位序查找与按值查找

本函数约定 i=0 返回头结点，i>=1 返回第 i 个数据结点

```
Node *GetElem(Node *L, int i) {
    if (i < 0) return nullptr;
    Node *p = L;
    for (int j = 0; p != nullptr && j < i; ++j)
        p = p->next;
    return p;
}
```

合法数据位序为 $1\sim n$；越过表尾返回空指针。开放 $i=0$ 是为了

方便寻找插入、删除位置的前驱，不表示头结点属于数据元素。按位序查找最坏时间为 $O(n)$。

按值查找：返回第一个匹配结点，失败返回空指针

```
Node *LocateNode(Node *L, int x) {
    Node *p = L->next;
    while (p != nullptr && p->data != x)
        p = p->next;
    return p;
}
```

2.3.2 插入：先保存旧后继，再接入新结点

在第 i 位插入前，先找到第 $i-1$ 个结点 `p`。新结点先接到旧后继，再让 `p` 接到新结点；两句赋值不能直接颠倒。

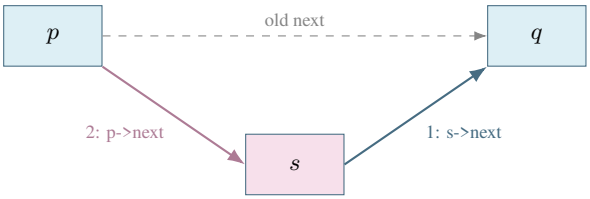


图 2.3: 插入分两步：s->next 接旧后继 q；p->next 再接 s。若先覆盖 p->next，可能丢失原来的后半链。

按位序插入：i=1 时前驱就是头结点

```
bool LinkedInsert(Node *L, int i, int x) {
    if (i < 1) return false;
    Node *p = GetElem(L, i - 1);
    if (p == nullptr) return false;
    Node *s = new Node{x, p->next};
    p->next = s;
    return true;
}
```

已知有效前驱时，修改链接只需 $O(1)$；从头寻找第 $i-1$ 个结点则最坏需要 $O(n)$。因此按位序插入的完整操作最坏仍为 $O(n)$。

C/C++ 表示

代码使用 new 申请结点、delete 释放结点；C 语言可用 malloc/free 实现同一结构，分配与释放方式要配对。

2.3.3 删除：先接好后链，再释放结点

按位序删除：先检查前驱和待删结点是否存在

```
bool LinkedDelete(Node *L, int i, int &x) {
    if (i < 1) return false;
    Node *p = GetElem(L, i - 1);
    if (p == nullptr || p->next == nullptr)
        return false;
    Node *q = p->next;
    x = q->data;
    p->next = q->next;
    delete q;
    return true;
}
```

删除的前提 释放 `q` 之后，不能再读取 `q->next`。只有待删结点的指针、却没有前驱时，普通单链表通常仍需从头找前驱；复制后继值再删后继的技巧不适用于尾结点，也不一定符合题目对结点身份的要求。

2.3.4 建立链表：头插与尾插

头插法：调用前 L 是空表，按输入逆序建立

```cpp
void BuildHead(Node *L, const int A[], int n) {
    for (int j = 0; j < n; ++j) {
        Node *s = new Node{A[j], L->next};
        L->next = s;
    }
}
```

尾插法：调用前 L 是空表，按输入原序建立

```cpp
void BuildTail(Node *L, const int A[], int n) {
    Node *rear = L;
    for (int j = 0; j < n; ++j) {
        Node *s = new Node{A[j], nullptr};
        rear->next = s;
        rear = s;
    }
}
```

输入 $1, 2, 3$，头插得到 $3 \to 2 \to 1$，尾插得到 $1 \to 2 \to 3$。两者均只对每个元素做常数次链接操作，建立时间为 $O(n)$。尾插若每次都从头找尾部，才会退化为 $O(n^2)$。

建立 n 个数据结点本身需要 $O(n)$ 存储；除结果链表外，工作指针只占 $O(1)$ 辅助空间。回答空间复杂度时，要说明是否把结果结构计入。

销毁普通无环单链表：包含头结点，结束后 L 为空

```cpp
void Destroy(Node *&L) {
    while (L != nullptr) {
        Node *q = L;
        L = L->next;
        delete q;
    }
}
```

2.4 双链表、循环链表与静态链表

2.4.1 双链表：同时维护前后链接

双链表的结点增加前驱指针 `prior`。已知结点后，可以在 $O(1)$ 时间访问它的直接前驱；按位序定位结点仍通常需要 $O(n)$ 时间。

在 p 之前插入 s：先设置新结点两侧，再修改旧结点链接

```
struct DNode {
    int data;
    DNode *prior, *next;
};
// p has a valid predecessor; s is a new node.
void InsertBefore(DNode *p, DNode *s) {
    s->prior = p->prior;
    s->next = p;
    p->prior->next = s;
    p->prior = s;
}
```

设旧前驱为 r，插入后必须同时满足 $r.next = s$、$s.prior = r$、$s.next = p$、$p.prior = s$。代码假设 `p->prior` 有效；无头结点链表在首元结点之前插入时，应另行修改头指针。

删除具有有效前驱和后继的数据结点 `p` 时，先接好两侧，再释放 `p`。下面的操作假设两个邻接结点都有效；普通非循环链表的尾结点后继为空，需另作判空处理。

双链表删除的链接操作

```
p->prior->next = p->next;
p->next->prior = p->prior;
delete p;
```

2.4.2 循环链表：终止位置变成头结点

带头结点的循环单链表，尾结点的 `next` 指回头结点。空表时头结点指向自己；遍历遇到头结点时结束，而不是等待空指针。

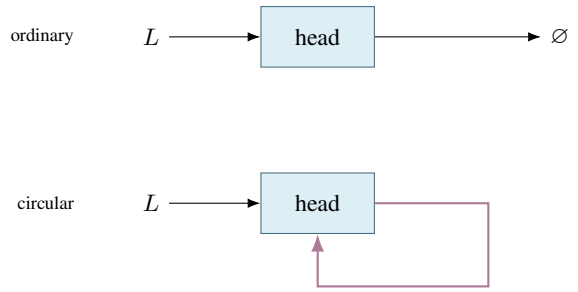


图 2.4: 带头结点的普通单链表与循环单链表：空表链接不同，遍历终止条件也不同。

结构约定	空表条件
普通单链表，带有效头结点	`L->next == nullptr`
循环单链表，带有效头结点	`L->next == L`
循环双链表，带有效头结点	`L->next == L` 且 `L->prior == L`

循环单链表保存尾指针 `rear` 时，可通过 `rear->next` 取得链头。对两个非空、无头结点的循环单链表，交换尾部的后继链接即可 $O(1)$ 拼接；只有头指针时，通常还要线性查找尾结点。带头结点的拼接还须处理多余头结点。

L 与 &L

`L` 是指针保存的结点地址；`&L` 是指针变量自身的地址。二者不能替换。

2.4.3 静态链表：用数组下标代替指针

静态链表把结点存放在数组中，用整数游标记录后继结点的数组下标。数组中相邻位置不一定是逻辑后继；它仍按链接查找，不因此获得按位序的常数时间随机访问。通常还需管理空闲结点链。

维度	顺序表	普通单链表
地址要求	元素连续	不同结点可分散
关系表示	由位置隐含	由 next 显式记录
按位序访问，最坏	$O(1)$	$O(n)$
无序按值查找，最坏	$O(n)$	$O(n)$
任意位序插删，最坏	$O(n)$，需移动	$O(n)$，含定位
额外结构空间	无需链接域	每个结点有指针域

选择结构的依据 频繁按位序访问，优先考虑顺序表。若经常在已定位的位置修改链接，链式结构较方便。选择结构时，应同时考虑定位成本、修改成本和存储开销。

2.5 基础巩固：一次扫描与原地逆置

2.5.1 顺序表删除所有值为 x 的元素

使用读下标 j 扫描原表，用 k 记录已保留元素个数。遇到不等于 x 的元素，就写到保留区末尾。这样不需要每删一个元素就搬移后半表。

保持原有相对次序，时间 O(n)，辅助空间 O(1)

```
void DeleteAll(SqList &L, int x) {
    int k = 0;
    for (int j = 0; j < L.length; ++j)
        if (L.data[j] != x)
            L.data[k++] = L.data[j];
    L.length = k;
}
```

整个过程中 $k \le j$，写入不会破坏尚未读取的元素；全部删除时 $k = 0$，全部保留时 $k = n$。

2.5.2 带头结点单链表原地逆置

prev 指向已逆置部分，cur 指向未处理部分。每轮先保存旧后继，再反转当前链接，随后推进工作指针。

保留头结点，时间 O(n)，辅助空间 O(1)

```
void Reverse(Node *L) {
    Node *prev = nullptr, *cur = L->next;
    while (cur != nullptr) {
        Node *nxt = cur->next;
        cur->next = prev;
        prev = cur;
        cur = nxt;
    }
    L->next = prev;
}
```

空表时循环不执行，`L->next` 仍为空；单结点表也能正常处理。最后要更新头结点的 `next`，不能把头结点当作数据结点一起逆置。

用条件检验结论

基础自检（补充题）

1. 顺序表随机存取是否表示按值查找一定为 $O(1)$?

2. 长度为 8 的顺序表，插入第 3 位、删除第 3 个元素，分别移动多少个元素?

3. 已知单链表前驱后删除其后继，修改链接要多久？若先按位序定位呢?

4. 带头结点循环单链表为空时，`L->next` 等于什么?

5. 头插法和维护尾指针的尾插法，建立 n 个结点各需要多少时间?

答案： 1. 否，无序表按值查找最坏 $O(n)$。2. 分别为 6 和 5。3. 修改链接 $O(1)$；含定位最坏 $O(n)$。4. 等于 `L`。5. 都是 $O(n)$；输入次序分别逆转和保持。